

Partie 1 : Classes et Objets (21 questions)

1. Quelle est la sortie du programme suivant ?

```
1 public class Init {  
2     static { System.out.print("S1"); }  
3     { System.out.print("I1"); }  
4     public Init() { System.out.print("C1"); }  
5     { System.out.print("I2"); }  
6     static { System.out.print("S2"); }  
7     public static void main(String[] args) {  
8         new Init();  
9     }  
10 }
```

- a) SIS21112C1
- b) I112S1S2C1
- c) S11112S2C1
- d) S1S2I1I2C1

2. Que se passe-t-il avec ce code ?

```
1 public class Test {  
2     private final int a;  
3     public Test() {  
4         System.out.print(a);  
5         a = 10;  
6     }  
7 }
```

- a) Affiche 0
- b) Affiche 10
- c) Erreur de compilation
- d) Exception à l'exécution

3. Quelle est la sortie du programme suivant ?

```
1 public class Chain {  
2     public Chain() { this(5); System.out.print("A"); }  
3     public Chain(int x) { this(x, 10); System.out.print("B"); }  
4     public Chain(int x, int y) { System.out.print("C"); }  
5     public static void main(String[] args) {  
6         new Chain();  
7     }  
8 }
```

- a) ABC
- b) CBA
- c) ACB
- d) BAC

4. Quelle est la sortie du programme suivant ?

```
1 public class Shadow {  
2     int x = 5;  
3     void test() {  
4         int x = 10;  
5         System.out.print(x + " " + this.x);  
6     }  
7 }
```

- a) 510
- b) 10 5
- c) 55
- d) 10 10

5. Quelle est la sortie du programme suivant ?

```
1 public class StaticInit {  
2     static int x = getValue();  
3     static int y = 5;  
4     static int getValue() { return y; }  
5     public static void main(String[] args) {  
6         System.out.print(x);  
7     }  
8 }
```

- a) 0
- b) 5
- c) null
- d) 0

6. Quelle est la sortie du programme suivant ?

```
1 public class Test {  
2     Test() { System.out.print("1"); }  
3     { System.out.print("2"); }  
4     Test(int x) { System.out.print("3"); }  
5     public static void main(String[] args) {  
6         new Test();  
7         new Test(5);  
8     }  
9 }
```

- a) 1213
- b) 2123
- c) 213
- d) 123

7. Que se passe-t-il avec ce code ?

```
1 public class PrivateConstructor {  
2     private PrivateConstructor() {}  
3     public static PrivateConstructor create() {  
4         return new PrivateConstructor();  
5     }  
6 }
```

```
5     }
6     public static void main(String[] args) {
7         PrivateConstructor pc = new PrivateConstructor();
8     }
9 }
```

- a) Compile et s'exécute
- b) Erreur : constructeur privé
- c) Warning seulement
- d) Exception à l'exécution

8. Quelle est la sortie du programme suivant ?

```
1 public class VarInit {
2     int x;
3     int y = x + 5;
4     public static void main(String[] args) {
5         VarInit v = new VarInit();
6         System.out.print(v.x + " " + v.y);
7     }
8 }
```

- a) 05
- b) 0 5
- c) 5 10
- d) Exception

9. Quelle est la sortie du programme suivant ?

```
1 public class MultiInit {
2     int a = 1;
3     { a = 2; }
4     int b = a * 3;
5     { b = 4; }
6     public static void main(String[] args) {
7         MultiInit m = new MultiInit();
8         System.out.print(m.a + " " + m.b);
9     }
10 }
```

- a) 1 3
- b) 2 4
- c) 1 4
- d) 2 6

10. Quelle est la visibilité par défaut d'une classe dans un fichier Java ?

- a) public
- b) protected
- c) private
- d) package-private

11. Quel mot-clé permet d'accéder à un membre depuis n'importe où ?

- a) public
- b) private
- c) protected
- d) (aucun)

12. Dans le code suivant, que peut-on accéder dans la classe B ?

```
1 package p1;
2 public class A {
3     private int a = 1;
4     int b = 2;
5     protected int c = 3;
6     public int d = 4;
7 }
8 package p2;
9 import p1.A;
10 public class B extends A {
11     void test() {
12         // Que peut-on accéder ici ?
13     }
14 }
```

- a) a, b, c, d
- b) b, c, d
- c) c, d
- d) d seulement

13. Une classe peut-elle être déclarée private ?

- a) Oui, toujours
- b) Non, jamais
- c) Oui, seulement si elle est interne
- d) Oui, seulement si elle est abstraite

14. Le mot-clé 'protected' permet l'accès :

- a) Dans le même package seulement
- b) Dans les classes dérivées seulement
- c) Dans le même package ET classes dérivées
- d) N'importe où

15. Quelle est la sortie du programme suivant ?

```
1 public class Test {
2     private static int count = 0;
3     public static int getCount() { return count; }
4     private static void increment() { count++; }
5     public static void main(String[] args) {
6         increment();
7         System.out.print(getCount());
8     }
9 }
```

- a) 0
- b) 1
- c) Erreur de compilation
- d) null

16. Quelle affirmation sur l'encapsulation est VRAIE ?

- a) Tous les attributs doivent être publics
- b) Tous les attributs doivent être privés
- c) Les getters/setters violent l'encapsulation
- d) L'encapsulation n'est pas importante en Java

17. Que se passe-t-il avec ce code ?

```
1 class Outer {  
2     private int x = 10;  
3     class Inner {  
4         void show() { System.out.print(x); }  
5     }  
6 }
```

- a) Erreur : x inaccessible
- b) Affiche 10
- c) Affiche 0
- d) Exception à l'exécution

18. Quelle est la différence entre 'private' et 'package-private' ?

- a) Aucune différence
- b) Private est plus restrictif
- c) Package-private est plus restrictif
- d) Ils sont synonymes

19. Une interface peut-elle avoir des méthodes privées ?

- a) Oui, toujours
- b) Non, jamais
- c) Oui, à partir de Java 9
- d) Oui, seulement si elle est marquée 'sealed'

20. Quelle est la sortie du programme suivant ?

```
1 public class Outer {  
2     private int x = 10;  
3     public class Inner {  
4         public void print() { System.out.print(x); }  
5     }  
6     public static void main(String[] args) {  
7         Outer.Inner inner = new Outer().new Inner();  
8         inner.print();  
9     }  
10 }
```

- a) 10
- b) 0
- c) Erreur de compilation
- d) null

21. **Quelle est la caractéristique d'une classe interne statique ?**

- a) Peut accéder aux membres non-statiques de la classe externe
- b) Doit être déclarée final
- c) N'a pas de référence implicite à la classe externe
- d) Ne peut pas être instanciée

Partie 2 : Méthodes et Surcharge (5 questions)

21. Quelle est la sortie du programme suivant ?

```
1 public class Test {  
2     static void method(int... nums) { System.out.print("varargs"); }  
3     static void method(int a, int b) { System.out.print("two"); }  
4     public static void main(String[] args) {  
5         method(1, 2);  
6     }  
7 }
```

- a) varargs
- b) two
- c) Les deux
- d) Erreur

22. Que se passe-t-il avec ce code ?

```
1 public class Ambiguity {  
2     void test(int a, long b) { System.out.print("1"); }  
3     void test(long a, int b) { System.out.print("2"); }  
4     public static void main(String[] args) {  
5         new Ambiguity().test(5, 5);  
6     }  
7 }
```

- a) Compile
- b) Erreur : ambiguïté
- c) Affiche 1
- d) Affiche 2

23. Quelle est la sortie du code suivant ?

```
1 public class ReturnTest {  
2     int getValue() {  
3         try { return 1; }  
4         finally { return 2; }  
5     }  
6     public static void main(String[] args) {  
7         System.out.print(new ReturnTest().getValue());  
8     }  
9 }
```

- a) 1
- b) 2
- c) 3
- d) Erreur

24. Les méthodes varargs :

- a) Doivent avoir exactement un paramètre varargs
- b) Le paramètre varargs doit être le dernier

- c) Peuvent avoir plusieurs paramètres varargs
- d) Ne peuvent pas être surchargées

25. Que se passe-t-il avec ce code ?

```
1 public class ConstructorOverload {  
2     ConstructorOverload() { this(5); }  
3     ConstructorOverload(int x) { this(); }  
4     public static void main(String[] args) {  
5         new ConstructorOverload();  
6     }  
7 }
```

- a) Compile
- b) Erreur : appel récursif
- c) StackOverflowError
- d) Exception

Partie 3 : Chaînes de caractères (8 questions)

26. Quelle méthode est utilisée pour comparer le contenu des chaînes ?

- a) ===
- b) equals()
- c) compare()
- d) match()

27. Quelle est la sortie du code suivant ?

```
1 StringBuilder sb = new StringBuilder("Hello");  
2 sb.append(" World").delete(5, 11).insert(5, ",");  
3 System.out.print(sb);
```

- a) Hello World
- b) Hello
- c) Hello,World
- d) Hello,

28. Quelle est la différence entre StringBuffer et StringBuilder ?

- a) StringBuffer est plus rapide
- b) StringBuilder est thread-safe
- c) StringBuffer est thread-safe
- d) Aucune différence

29. Quelle est la sortie du code suivant ?

```
1 String s = null;  
2 System.out.print(s + "test");
```

- a) nulltest
- b) test
- c) NullPointerException
- d) null

30. La méthode intern() :

- a) Crée une nouvelle chaîne
- b) Retourne la représentation canonique
- c) Convertit en majuscules
- d) Supprime les espaces

31. Quelle est la sortie du code suivant ?

```
1 String s = "Java";  
2 s = s.replace('a', 'o');  
3 System.out.print(s);
```

- a) Java
- b) Jovo
- c) JovoJovo
- d) Erreur

32. Quelle est la sortie du code suivant ?

```
1 String str = " Hello World ";  
2 System.out.print(str.trim().length());
```

- a) 13
- b) 11
- c) 15
- d) 10

33. Quelle est la sortie du code suivant ?

```
1 String s1 = "Hello";  
2 String s2 = s1.substring(0, 3);  
3 String s3 = "Hel";  
4 System.out.print(s2 == s3);
```

- a) true
- b) false
- c) Erreur
- d) null

Partie 4 : Tableaux (10 questions)

34. Quelle est la sortie du code suivant ?

```
1 int[] arr = new int[5];  
2 System.out.print(arr[0] + " " + arr.length);
```

- a) 0 5
- b) null 5
- c) 0 4
- d) null 4

35. Quelle est la sortie du code suivant ?

```
1 int[][] matrix = new int[3][];  
2 matrix[0] = new int[2];  
3 matrix[1] = new int[3];  
4 System.out.print(matrix[0].length + " " + matrix[1].length);
```

- a) 3 3
- b) 2 3
- c) 0 0
- d) Exception

36. Quelle déclaration est correcte ?

- a) `int[] arr = {1, 2, 3};`
- b) `int[] arr = new int[]{1, 2, 3};`
- c) `int[] arr = new int[] {1, 2, 3};`
- d) `int arr[] = new int[] {3};`

37. Quelle est la sortie du code suivant ?

```
1 int[] arr = {1, 2, 3, 4, 5};  
2 System.out.print(arr[arr[2]]);
```

- a) 3
- b) 4
- c) 5
- d) Exception

38. Que se passe-t-il avec ce code ?

```
1 Object[] objects = new String[3];  
2 objects[0] = "Hello";  
3 objects[1] = 123;
```

- a) Compile
- b) Erreur de compilation
- c) `ArrayStoreException`

d) NullPointerException

39. Quelle est la sortie du code suivant ?

```
1 int[] arr1 = {1, 2, 3};
2 int[] arr2 = arr1.clone();
3 arr2[0] = 100;
4 System.out.print(arr1[0] + " " + arr2[0]);
```

- a) 1 100
- b) 100 100
- c) 1 1
- d) 100 1

40. Quelle méthode trie un tableau ?

- a) Arrays.sort()
- b) Array.sort()
- c) Collection.sort()
- d) List.sort()

41. Que se passe-t-il avec ce code ?

```
1 int[] arr = new int[-5];
```

- a) Compile
- b) Erreur de compilation
- c) NegativeArraySizeException
- d) ArrayIndexOutOfBoundsException

42. Quelle est la sortie du code suivant ?

```
1 int[] nums = {1, 2, 3};
2 for (int n : nums) {
3     n = n * 2;
4 }
5 System.out.print(nums[0]);
```

- a) 1
- b) 2
- c) 3
- d) 6

43. Quelle est la sortie du code suivant ?

```
1 String[] arr = new String[2];
2 System.out.print(arr[0] + " " + arr[1]);
```

- a) null null
- b) ""
- c) Erreur
- d) Exception

Partie 5 : Énumérations (10 questions)

44. Quelle est la sortie du code suivant ?

```
1 enum Color { RED, GREEN, BLUE }  
2 System.out.print(Color.RED.ordinal());
```

- a) 0
- b) 1
- c) RED
- d) null

45. Les enums peuvent :

- a) Avoir des constructeurs
- b) Être étendues
- c) Être modifiées à l'exécution
- d) Implémenter l'héritage multiple

46. Que se passe-t-il avec ce code ?

```
1 enum Status {  
2     ACTIVE(1), INACTIVE(0);  
3     private int value;  
4     Status(int v) { this.value = v; }  
5 }
```

- a) Compile
- b) Erreur : constructeur privé
- c) Erreur : valeurs manquantes
- d) Warning

47. La méthode values() des enums :

- a) Retourne un tableau des constantes
- b) Retourne un String
- c) N'existe pas
- d) Retourne un List

48. Quelle est la sortie du code suivant ?

```
1 enum Direction {  
2     NORTH, SOUTH { public String toString() { return "S"; } }, EAST,  
3     WEST;  
4 }  
5 System.out.print(Direction.SOUTH);
```

- a) SOUTH
- b) S
- c) Erreur
- d) null

49. Les enums peuvent implémenter :

- a) Des interfaces
- b) Des classes abstraites
- c) Des classes concrètes
- d) Rien d'autre

50. Quelle est la sortie du code suivant ?

```
1 enum Size {  
2     SMALL, MEDIUM, LARGE;  
3     Size() { System.out.print(this.name() + " "); }  
4 }  
5 public class Test {  
6     public static void main(String[] args) {  
7         Size s = Size.MEDIUM;  
8     }  
9 }
```

- a) SMALL MEDIUM LARGE
- b) MEDIUM
- c) Rien
- d) Erreur

51. Quelle méthode compare deux enums ?

- a) ==
- b) equals()
- c) compareTo()
- d) Toutes

52. Que se passe-t-il avec ce code ?

```
1 enum Level {  
2     LOW, MEDIUM, HIGH;  
3     public static Level getDefault() {  
4         return MEDIUM;  
5     }  
6 }
```

- a) Compile
- b) Erreur : méthode non autorisée
- c) Erreur : static interdit
- d) Warning

53. Les enums sont implicitement :

- a) Serializable
- b) Cloneable
- c) Comparable
- d) Runnable

Partie 6 : Héritage (15 questions)

54. Quelle affirmation sur l'héritage est VRAIE ?

- a) Java supporte l'héritage multiple de classes
- b) Toutes les classes héritent d'Object
- c) Les classes finales peuvent être étendues
- d) Les constructeurs sont hérités

55. Quelle sortie pour new C() dans le code suivant ?

```
1 class A { A() { System.out.print("A"); } }  
2 class B extends A { B() { System.out.print("B"); } }  
3 class C extends B { C() { System.out.print("C"); } }
```

- a) ABC
- b) CBA
- c) A seulement
- d) C seulement

56. Le mot-clé super permet d'accéder :

- a) À la classe parente seulement
- b) À la classe parente et ses ancêtres
- c) À la classe Object seulement
- d) À toutes les classes du package

57. Quelle est la sortie du code suivant ?

```
1 class Parent {  
2     void show() { System.out.print("Parent"); }  
3 }  
4 class Child extends Parent {  
5     void show() { System.out.print("Child"); }  
6     void display() {  
7         show();  
8         super.show();  
9     }  
10 }
```

- a) ParentParent
- b) ChildChild
- c) ChildParent
- d) ParentChild

58. Quelle affirmation sur les constructeurs est FAUSSE ?

- a) super() doit être la première instruction
- b) this() et super() peuvent être ensemble
- c) Si aucun constructeur, un par défaut est fourni
- d) Les constructeurs sont hérités

59. Une classe peut étendre :

- a) Une seule classe
- b) Plusieurs classes
- c) Une classe et plusieurs interfaces
- d) Autant de classes que nécessaire

60. Le mot-clé final appliqué à une classe :

- a) Empêche l'héritage
- b) Empêche l'instanciation
- c) Rend la classe abstraite
- d) Rend tous les membres finals

61. Quelle est la sortie du code suivant ?

```
1 class A { int x = 1; }
2 class B extends A { int x = 2; }
3 A a = new B();
4 System.out.print(a.x);
```

- a) 1
- b) 2
- c) Erreur
- d) null

62. L'annotation @Override :

- a) Est obligatoire
- b) Vérifie la redéfinition à la compilation
- c) Change le comportement à l'exécution
- d) Est héritée

63. Quelle classe ne peut pas être étendue ?

- a) Une classe abstraite
- b) Une classe finale
- c) Une classe avec constructeur privé
- d) Une classe package-private

64. Que manque-t-il dans le constructeur Derived() ?

```
1 class Base {
2     Base(int x) { System.out.print(x); }
3 }
4 class Derived extends Base {
5     Derived() {
6         // Que manque-t-il ici ?
7     }
8 }
```

- a) super();

- b) `super(0);`
- c) `this();`
- d) Rien

65. L'héritage permet :

- a) La réutilisation de code
- b) L'encapsulation
- c) La surcharge d'opérateurs
- d) La gestion des exceptions

66. Que se passe-t-il avec ce code ?

```
1 class X { private void m1() {} }
2 class Y extends X { public void m1() {} }
```

- a) Redéfinition correcte
- b) Masquage
- c) Erreur : `m1()` privé dans X
- d) Surcharge

67. Une sous-classe peut-elle réduire la visibilité d'une méthode ?

- a) Oui
- b) Non
- c) Seulement si elle est finale
- d) Seulement si elle est statique

68. Quelle est la sortie du code suivant ?

```
1 class Parent {
2     static void method() { System.out.print("Parent"); }
3 }
4 class Child extends Parent {
5     static void method() { System.out.print("Child"); }
6 }
7 Parent p = new Child();
8 p.method();
```

- a) Parent
- b) Child
- c) Erreur
- d) Les deux

Partie 7 : Polymorphisme (9 questions)

69. Le polymorphisme permet :

- a) Une variable de type parent de référencer un objet enfant
- b) L'héritage multiple
- c) La surcharge uniquement
- d) La compilation conditionnelle

70. Quelle est la sortie du code suivant ?

```
1 class Animal { void sound() { System.out.print("Animal"); } }
2 class Dog extends Animal { void sound() { System.out.print("Dog"); } }
3 Animal a = new Dog();
4 a.sound();
```

- a) Animal
- b) Dog
- c) Erreur
- d) null

71. La liaison tardive (late binding) concerne :

- a) Les méthodes statiques
- b) Les méthodes final
- c) Les méthodes d'instance non-final
- d) Les constructeurs

72. Quelle est la sortie du code suivant ?

```
1 class A { void m() { System.out.print("A"); } }
2 class B extends A { void m() { System.out.print("B"); } }
3 class C extends B { void m() { System.out.print("C"); } }
4 A obj = new C();
5 obj.m();
```

- a) A
- b) B
- c) C
- d) Erreur

73. Le casting explicite est nécessaire quand :

- a) On assigne un parent à un enfant
- b) On assigne un enfant à un parent
- c) Toujours
- d) Jamais

74. Que se passe-t-il avec ce code ?

```
1 Object obj = "Hello";  
2 Integer num = (Integer) obj;
```

- a) Compile
- b) Erreur de compilation
- c) ClassCastException
- d) NullPointerException

75. L'opérateur instanceof vérifie :

- a) La compatibilité des types à la compilation
- b) La compatibilité des types à l'exécution
- c) L'égalité des objets
- d) La référence null

76. Que se passe-t-il avec ce code ?

```
1 class Shape { void draw() {} }  
2 class Circle extends Shape { void draw() {} }  
3 class Square extends Shape { void draw() {} }  
4 Shape s = new Circle();  
5 if (s instanceof Square) {  
6     Square sq = (Square) s;  
7 }
```

- a) Compile
- b) Erreur
- c) ClassCastException
- d) NullPointerException

77. Quelle affirmation est VRAIE ?

- a) Toutes les méthodes sont polymorphes
- b) Seules les méthodes non-final et non-private sont polymorphes
- c) Les méthodes statiques sont polymorphes
- d) Les constructeurs sont polymorphes

Partie 8 : Exceptions (10 questions)

78. Quelle classe est à la racine de la hiérarchie des exceptions ?

- a) Throwable
- b) Exception
- c) Error
- d) RuntimeException

79. Les **checked exceptions** doivent :

- a) Être cachées ou déclarées
- b) Être ignorées
- c) Être cachées seulement
- d) Être déclarées seulement

80. Que se passe-t-il avec ce code ?

```
1 try {  
2     throw new RuntimeException();  
3 } catch (Exception e) {  
4     System.out.print("Exception");  
5 } catch (RuntimeException e) {  
6     System.out.print("Runtime");  
7 }
```

- a) Exception
- b) Runtime
- c) Erreur : ordre incorrect
- d) Rien

81. Quelle affirmation sur **finally** est **VRAIE** ?

- a) Toujours exécuté
- b) Jamais exécuté si return dans try
- c) Optionnel
- d) Peut être sauté avec break

82. Quelle est la sortie du code suivant ?

```
1 try {  
2     return 1;  
3 } finally {  
4     return 2;  
5 }
```

- a) Retourne 1
- b) Retourne 2
- c) Erreur
- d) Exception

83. Les **Error** représentent :

- a) Des erreurs graves irrécupérables
- b) Des exceptions vérifiées
- c) Des exceptions d'exécution
- d) Des warnings

84. L'instruction **throw** :

- a) Déclare une exception
- b) Lance une exception
- c) Catch une exception
- d) Finally une exception

85. Que se passe-t-il avec ce code ?

```
1 void m1() throws IOException {}  
2 void m2() {  
3     try { m1(); }  
4     catch (IOException e) { }  
5     catch (Exception e) { }  
6 }
```

- a) Compile
- b) Erreur : catch inutile
- c) Erreur : ordre incorrect
- d) Warning

86. Les **assertions** servent à :

- a) Valider des préconditions
- b) Gérer les erreurs
- c) Remplacer les exceptions
- d) Debugger seulement

87. Que représente ce code ?

```
1 try (Scanner sc = new Scanner(System.in)) {  
2     // utilisation  
3 }
```

- a) try-with-resources
- b) Erreur de syntaxe
- c) Try classique
- d) Try catch implicite

Partie 9 : Questions avancées (10 questions)

88. Quel est l'output du code suivant ? (Héritage + Constructeurs)

```
1 class A {  
2     A() { System.out.print("A"); }  
3 }  
4 class B extends A {  
5     B() { System.out.print("B"); }  
6     B(int x) {  
7         this();  
8         System.out.print("B" + x);  
9     }  
10 }  
11 public class Main {  
12     public static void main(String[] args) {  
13         new B(5);  
14     }  
15 }
```

- a) ABB5
- b) ABB5B
- c) A BB5
- d) B5A B

89. Que produit ce code ? (Polymorphisme + Liaison tardive)

```
1 class Base {  
2     void display() { System.out.print("Base "); }  
3 }  
4 class Derived extends Base {  
5     void display() { System.out.print("Derived "); }  
6     void show() { System.out.print("Show "); }  
7 }  
8 public class Main {  
9     public static void main(String[] args) {  
10         Base b = new Derived();  
11         b.display();  
12         ((Derived)b).show();  
13     }  
14 }
```

- a) Base Show
- b) Derived Show
- c) Base Derived
- d) Erreur de compilation

90. Quel est le problème avec ce code ? (Exceptions + Héritage)

```
1 class Base {  
2     void process() throws IOException { }  
3 }  
4 class Derived extends Base {  
5     void process() throws Exception { }  
6 }
```

- a) IOException est plus spécifique
- b) Exception est plus générale → erreur
- c) Pas de problème
- d) Les deux doivent lancer la même exception

91. Quel est le résultat ? (Abstraction + Template Method)

```
1 abstract class Game {
2     abstract void initialize();
3     abstract void startPlay();
4     abstract void endPlay();
5
6     public final void play() {
7         initialize();
8         startPlay();
9         endPlay();
10    }
11 }
12 class Cricket extends Game {
13     void initialize() { System.out.print("Cricket Initialized "); }
14     void startPlay() { System.out.print("Cricket Started "); }
15     void endPlay() { System.out.print("Cricket Finished "); }
16 }
17 public class Main {
18     public static void main(String[] args) {
19         Game game = new Cricket();
20         game.play();
21     }
22 }
```

- a) Cricket Initialized Cricket Started Cricket Finished
- b) Erreur : classe abstraite non instanciable
- c) Rien
- d) Initialized Started Finished

92. Que se passe-t-il avec ce code ? (Polymorphisme + Surcharge)

```
1 class Calculator {
2     int calculate(int a, int b) { return a + b; }
3     double calculate(double a, double b) { return a * b; }
4 }
5 public class Main {
6     public static void main(String[] args) {
7         Calculator c = new Calculator();
8         System.out.print(c.calculate(2, 3) + " ");
9         System.out.print(c.calculate(2.0, 3.0));
10    }
11 }
```

- a) 5 6.0
- b) 6.0 5
- c) 5 5
- d) 6.0 6.0

93. Quel est l'output ? (Exceptions + Finally)

```
1 public class Test {
2     static String test() {
3         try {
4             System.out.print("try ");
5             return "return from try";
6         } finally {
7             System.out.print("finally ");
8         }
9     }
10    public static void main(String[] args) {
11        System.out.print(test());
12    }
13 }
```

- a) try finally return from try
- b) try return from try finally
- c) return from try try finally
- d) Erreur

94. Quel est le problème ? (Héritage multiple via interfaces)

```
1 interface A {
2     default void method() { System.out.print("A"); }
3 }
4 interface B {
5     default void method() { System.out.print("B"); }
6 }
7 class C implements A, B { }
```

- a) C doit implémenter method()
- b) Conflit de méthodes par défaut
- c) C devrait être abstract
- d) Pas de problème

95. Que produit ce code ? (Énumérations avancées)

```
1 enum Operation {
2     PLUS { double apply(double x, double y) { return x + y; } },
3     MINUS { double apply(double x, double y) { return x - y; } };
4     abstract double apply(double x, double y);
5 }
6 public class Main {
7     public static void main(String[] args) {
8         System.out.print(Operation.PLUS.apply(5, 3) + " ");
9         System.out.print(Operation.MINUS.apply(5, 3));
10    }
11 }
```

- a) 8 2
- b) 2 8
- c) PLUS MINUS
- d) Erreur

96. Quel est le résultat ? (Exceptions chaînées)

```
1 public class Test {
2     static void method1() {
3         throw new RuntimeException("Exception 1");
4     }
5     static void method2() {
6         try {
7             method1();
8         } catch (RuntimeException e) {
9             throw new RuntimeException("Exception 2", e);
10        }
11    }
12    public static void main(String[] args) {
13        try {
14            method2();
15        } catch (RuntimeException e) {
16            System.out.println(e.getMessage());
17            System.out.println(e.getCause().getMessage());
18        }
19    }
20 }
```

- a) Exception 1
Exception 2
- b) Exception 2
Exception 1
- c) Exception 1 Exception 2
- d) Erreur

97. Que se passe-t-il ? (Classes abstraites + Constructeurs)

```
1 abstract class Vehicle {
2     Vehicle() {
3         System.out.print("Vehicle ");
4         init();
5     }
6     abstract void init();
7 }
8 class Car extends Vehicle {
9     Car() {
10        System.out.print("Car ");
11    }
12    void init() {
13        System.out.print("Init ");
14    }
15 }
16 public class Main {
17     public static void main(String[] args) {
18         new Car();
19     }
20 }
```

- a) Vehicle Car Init
- b) Vehicle Init Car
- c) Init Vehicle Car
- d) Erreur

Fin du QCM